



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduzione a Javascript

Alcuni Framework Javascript

Fabio Vitali

Corso di laurea in Informatica
Alma Mater – Università di Bologna

Oggi parleremo di...

Alcuni framework a componenti:

- Angular.js
- Angular
- React
- Vue
- Svelte
- Web Components
- ... e altri minori



Caratteristiche dei framework a componenti

Interfacce sempre più complesse

- Molte pagine, molti stati, molte interazioni

Riuso e manutenibilità del codice

- Scopo: incapsulare HTML, stile e logica in blocchi riutilizzabili

Allineamento fra stato e interfaccia

- Il binding ed i modelli reattivi riducono i bug da mancata sincronizzazione tra visualizzazione e stato della computazione

Lavoro in team e design system

- Le librerie di componenti condivisi facilitano il riuso, la collaborazione e la coerenza visiva

Ecosistema di tool e performance

- CLI, bundler, dev-server, ottimizzazioni di build rendono scalabili anche progetti grandi





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Angular.js

Angular.js

- Nasce nel 2009 come progetto privato di un dipendente Google e un suo amico. Diventa un progetto Open Source nel 2010, ufficialmente supportato da Google. Il team di sviluppo era composto da tre persone.
- E' un framework per la semplificazione della realizzazione di Single Page Application, basato sui pattern Model-View-Controller (MVC) e Model-View-ViewModel (MVVM).
- Nel 2015 evolve al di là e al di fuori del motore di template e diventa una vera e propria piattaforma applicativa.
- Ogni parte di una pagina web può diventare la *view* di un *model*, e via Javascript si crea il *controller* che ne gestisce bi-direzionalmente la creazione e lo sviluppo.



Angular.js vs. Angular

- Alla fine del 2014, gli sviluppatori di Angular fecero un annuncio:
La nuova release di AngularJS sarà drasticamente differente, non compatibile all'indietro e senza un percorso di evoluzione dalla versione precedente.
- Per sottolineare il cambiamento, si richiese che d'ora in poi si intendesse con AngularJS la versione 1.x fin lì utilizzata, e con Angular le versioni successive, senza fare riferimento alla versione utilizzata.
- Oggi AngularJS è fuori supporto ed è usato solo come modello concettuale o per rari casi di manutenzione di progetti legacy.
- Angular non è più un sofisticato motore di template HTML basato su Model, View e Controller, ma un ambiente di produzione di applicazioni web basato sul concetto modulare di componente che include ***anche*** dei template HTML per la loro presentazione a schermo.

Angular.js vs. Angular

- Angular ha una parabola interessante: nata come side project di programmatori di Google, ha avuto un grande successo di immagine per qualche anno, poi offuscata dalla nascita di React e Vue.
- ***Violentemente opinionato***, Angular richiede un po' di tempo per abituarcisi, e forma e vincola grandemente il progettista al modello applicativo implicito nella libreria.
- ***Dependency injection*** e ***unit testing*** rappresentano le basi più astratte su cui è stata fondata una libreria di template, che si è evoluta rapidamente.
- Gli stessi sviluppatori hanno ritenuto questi due concetti molto più importanti del ***templating*** e del ***Model-View-Controller***, tanto da lasciarli indietro nelle nuove versioni e basarle su concetti completamente diversi.



React.js

- Nasce nel 2011 all'interno di Facebook, e ha la prima installazione importante con Instagram nel 2012. Diventa open source nel 2013.
- Inizialmente solo un motore di template, poi un application framework per i browser, infine con React Native (2015) un ambiente di progettazione integrato che comprende anche gli ambienti mobili.
- React Native continua ad usare Javascript come motore fondamentale, ma non usa HTML o il DOM come destinazione dei comandi, ma view native del sistema operativo di destinazione.



Vue.js

- Nato nel 2014 all'interno di Google come costola semplificata di Angular:
 - "I figured, what if I could just extract the part that I really liked about Angular and build something really lightweight without all the extra concepts involved?" (Evan You)
- Possiede template, sintassi handlebar, espressioni Javascript arbitrarie, binding bidirezionale, direttive (chiamate *componenti*), una struttura molto semplice di applicazione.
- Non possiede: scope, dependency injection, controller.



WebComponents

Prendendo ispirazione dai sistemi a componenti come Angular, React e Vue, anche la sintassi standard di HTML ha introdotto i web component.

- *Custom element*: posso estendere il vocabolario degli elementi di HTML con quelli che più mi piacciono.
- *Shadow DOM*: posso creare un mini-DOM protetto a cui non si applicano script, stili e eventi del documento principale
- *HTML template*: posso associare ad un custom element un frammento HTML che va automaticamente a rimpiazzare il custom element di riferimento.



Altri framework leggeri

- Preact
 - Compatibile con l'ecosistema React
 - API molto simili, bundle molto più piccolo
 - Utile per widget e progetti dove il peso è critico
 - Svelte / SvelteKit
 - Framework compilato senza virtual DOM a runtime
 - Moduli con markup, stile e logica insieme
 - Routing, SSR, API endpoints e deployment attraverso SvelteKit
 - Alpine.js
 - *Tailwind per JavaScript*: stato e comportamento direttamente nell'HTML
 - La logica è quasi tutta lato client, con piccole espressioni JS negli attributi
 - Ideale per dare interattività a pagine già rese dal server
 - htmx
 - Estende l'HTML con attributi dal comportamento predefinito.
 - Richiede un'applicazione server-side che invia frammenti HTML (non JSON)
 - Utile per applicazioni server-side classiche
-

Opinionatedness

Angular	AngularJS	Vue	Svelte	React/Preact	Web Components
completo, fortemente prescrittivo, obblighi su struttura, pattern, tool	molto "magico" e strutturato, ma meno rigido di Angular	molte convenzioni (SFC, router ufficiale, store, ecc.) ma si possono usare in maniera progressiva	ha sintassi propria e un ecosistema di riferimento (SvelteKit), ma è molto meno dogmatico di Angular o Vue.	view-first: poche opinioni, molte scelte delegate al programmatore	quasi zero convenzioni: solo API di piattaforma; tutto il resto dipende dal programmatore



Concetti comuni

- **Componente**: un frammento di interfaccia riutilizzabile (bottone, card, form, pagina).
 - Incapsula markup, stile e logica in un'unità coerente, con un nome e una responsabilità precisa.
 - Riceve dati in ingresso (props, attributi, ecc.) e aggiorna il suo output quando i dati cambiano.
 - Comunica con l'esterno tramite eventi o callback (click, submit, change...).
- **Template**: un frammento di HTML (o JSX, o simili) che descrive la struttura visiva del componente.
- **Interpolazione**: inserire nel template valori dinamici dell'ambiente
 - se abbiamo in scope la variabile `user`, allora `{{user.name}}` visualizza nel template il valore del suo campo `name`.
- **Props**: dati che arrivano dall'esterno del componente (dal genitore o dalla pagina). Sempre in sola lettura.
- **Stato**: dati interni al componente (contatori, flag, input), che cambiano nel tempo e obbligano a ri-renderizzare la vista.
- **Pagina**: spesso è solo un componente “grande” che contiene componenti più piccoli (header, sidebar, lista, footer...).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Angular.js

Angular.js: model-view-controller

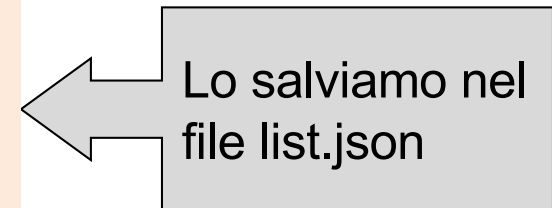
- Il *model* è una struttura dati (tipicamente espressa in JSON o come oggetto Javascript) che può esistere nella pagina web, essere caricata dinamicamente via Ajax, o essere calcolata a seguito di computazioni client-side.
- La *view* è un frammento HTML nella pagina principale, che viene decorato con speciali attributi (che iniziano con **ng-**) e placeholder (con la sintassi **{{nome}}**).
- Il *controller* è il codice Javascript che lega la view al model.
- Il *module* è il contenitore di tutte le parti che costituiscono un'applicazione AngularJS.
- La *directive* è una struttura di markup (elemento, attributo, classe) a cui associare un comportamento specifico.



Angular.js: il model

- Il *model* è una struttura dati (tipicamente espressa in JSON o come oggetto Javascript) che può esistere nella pagina web, essere caricata dinamicamente via Ajax, o essere calcolata a seguito di computazioni client-side.
- Ad esempio:

```
{
  "users" : [{
    "name": "Fabio Vitali",
    "tel": "051 2094872",
    "studenti": 120
  }, {
    "name": "Angelo Di Iorio",
    "tel": "051 2094889",
    "studenti": 60
  }]
}
```



Angular.js: la view

- La *view* è un frammento HTML nella pagina principale, che viene decorato con speciali attributi (che iniziano con **ng-**) e placeholder (con la sintassi **{{nome}}**).
- Nella view viene indicato il controller da utilizzare nel frammento:

```
<html>
  <body ng-app="simpleApp">
    <h1>Un semplice test Angular</h1>
    <table border="1" ng-controller="listCtrl">
      <tr>
        <th>Nome</th><th>Tel</th><th>studenti</th>
      </tr>
      <tr ng-repeat="person in list">
        <td>{{person.name}}</td>
        <td>{{person.tel}}</td>
        <td align='right'>{{person.studenti}}</td>
      </tr>
    </table>
  </body>
</html>
```

Angular.js: il controller

- Il module è il contenitore di tutte le parti che costituiscono un'applicazione AngularJS. Il controller lega la view al model.
- Il controller esplicitamente fa riferimento agli oggetti che vuole vedere. Questa si chiama *dependency injection*.
- In questo caso facciamo riferimento ai seguenti oggetti:
 - \$scope: lo scope di visibilità della view
 - \$http: l'oggetto che si occupa di comunicazioni via Ajax.

```
var simple = angular.module('simpleApp', []);
simple.controller('listCtrl', function($scope, $http){
    $http.get('list.json').success(function(data) {
        $scope.list = data.users;
    });
});
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Angular

Da AngularJS a Angular

Angular evolve da AngularJS rinunciando ad alcuni dei suoi concetti più importanti e rivoluzionandone altri:

- Si passa da MVVM a CBA (Component-Based Architecture)
- Mentre AngularJS prende una pagina HTML già organizzata e la arricchisce con contenuti dinamicamente generati da Javascript su direttive e template, Angular invece costruisce da zero il DOM finale a partire da frammenti HTML e CSS indipendenti tra loro, inseriti ed integrati in appositi moduli JS/Typescript.
- La sintassi AngularJS usa il prefisso ng- per ogni direttiva nel codice HTML. Angular usa decorator TypeScript (`click`) invece che ng-click, e però `*ngFor` invece che ng-repeat).
- Si abbandonano i controller e gli scope espliciti, sostituiti dall'incapsulamento dato dai componenti.
- Si adotta il linguaggio Typescript, compilato in Javascript per l'esecuzione.



Concetti di Angular

- *Moduli*: **NON** quelli di Javascript, ma insiemi logici di componenti, direttive e servizi, pensati per essere incapsulati e riutilizzabili come blocchi funzionali dell'applicazione.
 - *Direttive*: componenti speciali che cambiano il comportamento del markup e il DOM che viene generato. Introdotte dal decoratore `@Directive`.
 - Direttive strutturali: cambiano il contenuto del DOM: ad es. `*ngFor`, `*ngIf`
 - Direttive di attributo: arricchiscono il DOM con comportamenti: ad es. `[ngModel]`.
 - *Componenti*: un tipo di direttiva a cui corrisponde un template HTML e un selettore (i.e. un nuovo elemento di markup) che andrà a sostituire nel DOM finale. Introdotto dal decoratore `@Component`.
 - *Routing*: l'applicazione può passare da uno macro stato all'altro (ad esempio elenco articoli, vista articolo, vista carrello, ecc.) attraverso un meccanismo di routing interno che modifica anche l'URI visibile e gestisce scoping e passaggio dati da parte a parte dell'applicazione
 - **N.B.:** I *decoratori* sono annotazioni del codice nate in Python per semplificare la specifica di funzioni e classi, e sono stati adottati in seguito da TypeScript. Ne riparlamo.
-

Angular setup e bootstrap

- AngularJS ha un documento HTML che contiene le direttive fondamentali per l'attivazione della applicazione, ng-app e ng-controller.
- Angular invece ha una Command Line Interface (CLI) attraverso cui creo un progetto, che automaticamente genera una struttura di directory e di file predefiniti, di cui andare a modificare solo quelli rilevanti.
- Ogni componente viene posto in una directory indipendente che contiene un frammento HTML, una dichiarazione di classe in Typescript, e uno o più file CSS associati, ottenendo una totale autonomia ed incapsulamento del codice del componente



Angular: un esempio

Creiamo un nuovo componente

- da CLI:

```
> ng generate component hello-world
```

- la CLI crea una cartella pre-popolata:

```
src/app/hello-world
```

- `hello-world.component.ts` -> codice Typescript del component
- `hello-world.component.css` -> codice CSS del component
- `hello-world.component.html` -> template HTML del component
- `hello-world.component.spec.ts` -> codice di test del component

- I template degli altri componenti possono usare il nuovo tag:

```
<app-hello-world> ... </app-hello-world>
```

- I template possono contenere direttive Angular e interpolazione di dati passati dall'applicazione.

```
<p>Hello {{name}}! </p>
```

Un esempio

app-component.html

```
<h1>
  <app-hello-world>
    <p>Benvenuto</p>
  </app-hello-world>
</h1>
```

hello-world.component.html

```
<div>
  <p>Hello {{name}}!</p>
  <ng-content></ng-content>
</div>
```

hello-world.component.ts

```
@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-
world.component.html',
  styleUrls: ['./hello-world.component.css']
})

export class HelloWorldComponent
  implements OnInit {

  name:string;

  constructor() {
    this.name="Fabio"
  }

  ngOnInit() { }
}
```




ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

React.js

Principi base di REACT (1)

Virtual DOM

- L'applicazione REACT non cambia il DOM, ma cambia una copia del DOM sotto il controllo della libreria. Questa controlla propagazioni e effetti collaterali e aggiorna quando serve il DOM vero modificando ciò che vede l'utente (*reconciliation*).

One-way data binding

- Ogni elemento del DOM è associato ad uno store, e quindi ogni modifica viene effettuata allo store. Lo store chiederà al Virtual DOM di rieseguire il rendering dell'elemento associato, e da lì propagata al DOM.

Universal rendering

- L'uso del Virtual DOM implica che la mia applicazione non debba per forza interagire con elementi HTML di una pagina web, né che debba per forza risiedere su un browser.
- Attraverso renderer diversi (e.g., React Native) posso ottenere la stessa applicazione funzionante nativamente su device mobili. Attraverso Next.js posso creare Server-Side Rendering (SSR) precompilati per ottimizzare SEO e velocità dell'applicazione.



Principi base di REACT

JSX

- Un mix di javascript, XML e CSS per semplificare la realizzazione di template HTML che includono computazioni e calcolo di espressioni Javascript / Typescript.
- In particolare la coesistenza di HTML e JS permette di semplificare la coesistenza dei separatori di stringhe (virgolette) e nuovo markup
- Babel è il transpiler di React che controlla la coesistenza.

Components

- Un'unità indipendente di codice, markup e stile, richiamabile con markup inventato e riutilizzabile e configurabile via *props*.



React.js: modello di processo (1)

- Un component in React è una funzione che restituisce codice HTML o JSX.
- La funzione `render()` contiene il template da utilizzare per la sua visualizzazione.
- Per semplificare la vita del programmatore, il programma è scritto secondo una sintassi speciale, JSX, che mescola comandi Javascript e markup HTML

```
import { createRoot } from 'react-dom/client';

function Hello( {name}) {
  return <h1>Hello, {name} || "world"!</h1>;
}

const root = createRoot(document.getElementById('root'));
root.render(<Hello name="Fabio" />);
```

- Il componente React a questo punto permette una struttura di markup riutilizzabile basato su nuovi tag:

```
<body>
  <div id="root"></div>
  <p>Come va? Tutto bene?</p>
</body>
```

React.js: modello di processo (2)

- JSX è un formato XML che mescola in maniera appropriata codice Javascript, frammenti HTML e elementi XML creati ad hoc (i *componenti*).

```
import { createRoot } from 'react-dom/client';

function Hello({name}) {
  return <h1>Hello, { name || "world"}</h1>;
}

function Many() {
  return (
    <div>
      <Hello name="Fabio" />
      <Hello name="Alice" />
      <Hello />
    </div>
  );
}

const root = createRoot(document.getElementById('root'));
root.render(<Many />);
```

React.js: JSX e Babel

- JSX è un'estensione di Javascript che ammette al suo interno anche frammenti non protetti di codice di markup.
- L'interprete Babel esamina sistematicamente e ricorsivamente il codice JSX.
 - Ogni espressione JavaScript viene valutata ed eseguita.
 - Per ogni elemento per cui esiste una funzione con lo stesso nome, viene eseguita la funzione e l'output sostituisce l'elemento.
 - Per ogni elemento per cui esiste una classe con lo stesso nome, viene eseguita la funzione render() e l'output sostituisce l'elemento.
- Appena non ci sono più sostituzioni da fare, l'interpretazione è conclusa. Il metodo `ReactDOM.render()` inserisce nel nodo specificato il codice JSX interpretato.



Preact

- Stesso modello di React: componenti, JSX, props, stato.
- Dimensioni ridotte del bundle (pochi kB): adatto per widget, pagine leggere, progetti dove il peso è critico.
- API molto simili: spesso basta usare preact/compat per far girare codice React senza modifiche.
- Comunità più ridotta e compatibilità con React ottenuta sempre a posteriori.

```
import { render } from 'preact';

function Hello({ name }) {
  return <h1>Hello, {name}!</h1>;
}

render(<Hello name="Fabio" />, document.getElementById('root'));
```





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Vue.js

Vue.js

- **Interpolazione:** Una struttura unica mantiene tutti i dati visibili all'applicazione in un'unica struttura dati. Ogni formula di interpolazione nel documento accede a questa struttura dati in maniera diretta.
- **Reattività:** ogni interazione dell'utente permette di attivare una funzione di callback che interagisce direttamente con i dati e aggiorna il DOM subito.
- **Progressive:** dal semplice script in un HTML tradizionale ad una toolchain completa con routing e store.

```
<body>
  <div id="app">
    <button @click="count++">
      Hai cliccato {{count}} volte.
    </button>
  </div>

  <script type="module" >
    import { createApp } from 'vue'
    createApp({
      data() {
        return { count: 0 }
      }
    }).mount('#app')
  </script>
</body>
```

Vue.js: Single File Component

```
<script setup>
let count = 0

function increment() {
  count++
}
</script>

<template>
  <button @click="increment">
    Hai cliccato {{ count }} volte.
  </button>
</template>

<style scoped>
  button {
    color: red;
  }
</style>
```

Come tutti i framework, anche Vue suggerisce una scomposizione della pagina in componenti isolati ed indipendenti

Il Single File Component (SFC) contiene:

- I template HTML da inserire nel DOM
- Gli stili CSS locali da non condividere
- I dati su cui effettuare le operazioni
- I metodi visibili solo localmente



Componenti interpolazione

- Un componente può essere associato ad un elemento di markup.

```
import { createApp } from 'vue'
const app = createApp({})

app.component('modal', {
  props: ['id', 'title'],
  template: `
    <div class="modal" tabindex="-1" :id="id" >
      <div class="modal-header">
        <h5 class="modal-title">{{title}}</h5>
        <button type="button" class="btn-close"></button>
      </div>
      <div class="modal-body">
        <slot></slot>
      </div>
      <div class="modal-footer">
        <button type="button">Close</button>
      </div>
    </div>`
})
```

```
<modal id="info" title="Informazioni">
  <p>Un po' di informazioni</p>
</modal>
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Svelte

Svelte

- Libreria del 2016, circa 2kb, richiede build, supporta Javascript e Typescript.
- Basato su un framework per applicazioni chiamato SvelteKit e un linguaggio, *svelte*, simile a JSX.

```
<script>
  let count = 1;
  $: doubled = count * 2;
</script>

<p>Valore: {count}, il doppio vale {doubled}</p>
<button on:click={() => count = count + 1}>
  Incrementa valore
</button>
```

- Le variabili globali sono il modello dell'applicazione (lo stato)
- Lo statement \$: introduce variabili reattive, che cambiano automaticamente di valore quando cambiano le dipendenze.



Svelte

- Un componente è un file separato scritto in svelte

```
// file 'Modal.svelte'
<div class="modal" tabindex="-1" style="{hidden?'display:none;':''}">
  <div class="modal-header">
    <h5 class="modal-title">{title}</h5>
    <button type="button" class="btn-close"></button>
  </div>
  <div class="modal-body">
    <slot></slot>
  </div>
  <div class="modal-footer">
    <button type="button" on:click="{toggle}">Close</button>
  </div>
</div>
<script>
  export let title = "Default";
  export let hidden = true;
  function toggle() {
    hidden = !hidden;
  }
</script>
```

```
// file 'App.svelte'
<script>
  import Modal from './Modal.svelte';
</script>

<main>
  <h1>Hello Modal!</h1>
  <Modal title="my title" hidden>
    <p>Hello World</p>
  </Modal>
</main>
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Web Component

WebComponents

Un componente è una classe associata ad un elemento di markup con prefisso e registrata nell'array globale `customElement`.

- Il costruttore della classe genera un frammento HTML (ad esempio prendendo un template ed interpolandolo con variabili di ambiente)
- Il template interpolato viene posto in una parte "nascosta" del DOM detta shadow DOM.
- Quando voglio visualizzare il componente uso il metodo `attachShadow()` che inserisce lo shadowDOM in un punto stabilito del DOM vero.
- La separazione tra pagina HTML e componenti è facoltativa.
- L'uso di template è facoltativo.
- Il prefisso è obbligatorio.
- L'uso dello shadow DOM è facoltativo.
- Si usa `<slot>` per annidare sotto-componenti



WebComponents

```
<script>
customElements.define('my-modal',
  class extends HTMLElement {
    constructor() {
      super();
      let tp1 = document.getElementById('modalTp1').content;
      const shadowRoot = this.attachShadow({mode: 'open'});
      shadowRoot.appendChild(tp1.cloneNode(true));
    }
  }
);
</script>
<template id="modalTp1">
  <div class="modal" tabindex="-1">
    <div class="modal-header">
      <h5 class="modal-title"><slot name="title"></slot></h5>
      <button type="button" class="btn-close"></button>
    </div>
    <div class="modal-body">
      <slot></slot>
    </div>
    <div class="modal-footer">
      <button type="button">Close</button>
    </div>
  </div>
</template>
```

```
<my-modal id="m1" title="Hello world">
  <p>Some content</p>
</my-modal>
```



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HTMX e Alpine

HTMX

- Nel 2023 è il secondo arrivato nella categoria framework Front-end frameworks dopo React nel concorso JavaScript Rising Stars.
- Supporta AJAX, CSS Transitions, WebSockets and Server-Sent Events in HTML, usando solo attributi (un po' come fa Bootstrap), creando interfacce semplici e potenti manipolando solo markup.
- HTMX ripropone lo scambio di dati attraverso strutture HTML, e non JSON da convertire client-side, che rende più semplice il rendering e sposta sul server le computazioni presentazionali.
- HTMX è piccolo (circa 14kb), non ha dipendenze, è estensibile, ed è MOLTO più piccolo di altre librerie mastodontiche.

```
<button
  hx-post="/getData"
  hx-trigger="click"
  hx-target="#targetDiv"
  hx-swap="outerHTML">
  Carica dati
</button>
```

```
<button
  hx-delete="/account"
  hx-confirm="Are you sure?">
  Cancella Account
</button>
<button
  hx-put="/account"
  hx-confirm="Are you sure?">
  Aggiorna dati
</button>
```

Alpine

- Il jQuery del web moderno
- Un'unica libreria da aggiungere in <head> e nessuna build
- 15 attributi, 6 proprietà e 2 metodi

```
<div class="modal" tabindex="-1" id="modal1"
  x-data="{hidden: true}"
  x-bind:style="{hidden?'display:none;':''}" >
  <div class="modal-header">
    <h5 class="modal-title">{title}</h5>
    <button type="button" class="btn-close"></button>
  </div>
  <div class="modal-body"
    x-html="(await axios.get('/modalContent.html')).data"
  ></div>
  <div class="modal-footer">
    <button type="button"
      x-on:click="{hidden = !hidden}"
    >Close</button>
  </div>
</div>
```





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Ecosistemi per i framework a componenti

Il nuovo ciclo di sviluppo delle app web

- Edit-Save-Reload
 - Il vecchio modello web: un file di markup HTML, un file di stili CSS, un file di script JavaScript.
 - Sviluppo: carico un file in un editor, lo modifico, lo salvo, vado sul browser, ricarico la pagina
 - Deploy = trasferire gli stessi file sul server di produzione
- Build pipeline
 - Applicazioni sempre più grandi, strumenti sempre più sofisticati, riuso, typescript, componenti, ecc. rendono il vecchio modello inutilizzabile
 - Oggi parliamo di una build pipeline in cui i file sorgenti vengono passati attraverso una sequenza di strumenti che generano l'app eseguibile.
 - L'ecosistema è l'insieme di tutti i tool che intervengono tra il sorgente e l'oggetto finito di cui fare deploy
 - Ogni ecosistema ha tool specifici del framework e tool condivisi con altri framework



Alcuni concetti

- **CLI (Command Line Interface):** programma a riga di comando che crea progetti, genera file, lancia dev server e build (ng, npm create, vite, ...).
- **Dev server:** piccolo server locale che serve l'app, osserva i file e ricarica automaticamente la pagina (o i soli moduli) quando salvi.
- **Transpiling:** traduzione da un linguaggio sorgente a uno che il browser capisce: TypeScript → JavaScript, JSX → JavaScript, ecc.
- **Bundler:** Strumento che prende molti file (JS, CSS, immagini, font) e li impacchetta in pochi asset ottimizzati per il browser.
- **Build:** Processo che esegue transpiling, bundling, minificazione, ottimizzazioni: produce i file “di produzione” da mettere online.
- **Test & lint:** Controlli automatici su correttezza e stile del codice (unit test, test di integrazione, linting per regole e convenzioni).
- **Deploy:** Pubblicazione finale dei file di build su un server o una piattaforma cloud



La pipeline di Angular

La pipeline di Angular passa tutta attraverso la Angular CLI da riga di comando

- `ng new app-name` crea un progetto: struttura cartelle, opzioni di TypeScript, test, config, build.
 - `ng generate component/service/module` aggiunge all'app strutture usando un'organizzazione di file pre-impostata.
 - `ng serve` avvia il dev server: compila al volo, apre il browser, ricarica dopo ogni salvataggio.
 - transpiling: Angular **richiede** TypeScript. La transpilazione in JavaScript è parte fondamentale del build.
 - Bundler: Durante `ng serve` e `ng build` angular usa un bundler integrato (webpack / esbuild) per gestire import, asset, riorganizzazione del codice.
 - `ng build --configuration production` produce in `dist/` i bundle ottimizzati:
 - `ng test` esegue i test di modulo, `ng lint` controlla lo stile e i problemi comuni nel codice (usando ESLint).
 - L'output di `ng build` si carica su cloud o server
-

La pipeline di React

La CLI storica di React è `Create React App`, ma è ormai stata soppiantata da `vite`, una toolchain generica (funziona con React, Vue, Svelte, ecc.) creata da Evan You, l'autore di Vue, nel 2020.

- Vite non è integrata con React, si installa come pacchetto npm sulla propria macchina.
- `npm create vite@latest nome-app -- --template react` crea il progetto, struttura le cartelle, crea l'entry point `main.jsx`, il file di configurazione di Vite (`vite.config.js`), specifica se si usa TypeScript (opzionale: `--template react-ts`).
- I componenti vengono creati a mano semplicemente creando file `.jsx` e importandoli dove serve.
- `npm run dev` avvia il dev server: usa esbuild per servire i moduli “quasi direttamente” dal file system, con hot module replacement molto veloce.
- Transpiling: Vite delega a esbuild/plugin la trasformazione di JSX (e TS) in JavaScript standard.
- Bundler: `npm run build` usa Rollup per creare bundle ottimizzati (code splitting, minify, hashing dei file).
- `npm run preview` simula in locale l'app come sarà servita in produzione, usando l'output di build.
- L'output di `npm run build` (cartella `dist/`) si carica su hosting o cloud.

La pipeline di Vue

La pipeline di Vue si basa su uno scaffold ufficiale (= struttura vuota di progetto) chiamato `create-vue`. Questo scaffold usa già Vite in maniera appropriata.

- `npm create vue@latest` crea il progetto: struttura le cartelle, genera l'entry point `main.js/main.ts`, configura `vite.config.js` e chiede se si vogliono TypeScript, un router, uno store (Pinia), e strumenti ulteriori come ESLint, ecc.
 - I componenti e le altre strutture (layout, pagine, store) si creano a mano come file `.vue`: i Single File Components (SFC) contengono `<template>`, `<script>` (spesso con `script setup`) e `<style>`, più eventuali file JS/TS di supporto.
 - `npm run dev` avvia il dev server di Vite con hot module replacement rapido. Il compilatore di Vue trasforma i `.vue` (e l'eventuale TypeScript) in moduli JavaScript standard.
 - `npm run build` usa Vite + Rollup per generare bundle ottimizzati nella cartella `dist/`, che poi si pubblica su hosting statico o piattaforme cloud.
-

La pipeline di Svelte

La pipeline di Svelte passa attraverso il tool SvelteKit, anch'esso basato su Vite.

- `npm create svelte@latest nome-app` crea il progetto: struttura cartelle (`src/routes`, `src/lib`), genera l'entry point, configura `svelte.config.js` e `vite.config.js`, chiede se usare TypeScript, ESLint, test E2E, ecc.
 - Componenti e altre strutture (pagine, layout, componenti condivisi) si creano a mano con file `.svelte` dentro `src/routes` (per le pagine) e `src/lib` (per i componenti riusabili), più eventuali file JS/TS di supporto.
 - `npm run dev` avvia il dev server (Vite) con hot module replacement rapido; il compilatore di Svelte trasforma i `.svelte` in JavaScript e CSS ottimizzati.
 - `npm run build` esegue la build tramite SvelteKit/Vite, usando l'adapter configurato (Node, statico, ecc.) e produce l'output pronto per il deploy su hosting o piattaforme cloud.
-

La pipeline di WebComponents

Non esiste una CLI standard o una pipeline unica per i Web Components: sono API di piattaforma. Nei casi più semplici si lavora **senza build**:

- si scrivono classi ES6 che estendono `HTMLElement`,
 - si usano moduli nativi con `<script type="module">`,
 - si serve tutto con un semplice dev server (anche `python -m http.server`).
 - Per progetti più grandi si adotta una pipeline “simile ai framework”:
 - CLI generiche (Vite, esbuild, webpack) per dev server e build;
 - opzionale transpiling (TypeScript → JS);
 - bundler per raggruppare moduli e asset;
 - eventuali librerie come Lit o Stencil che aggiungono sintassi di templating, tooling e comandi propri.
 - In tutti i casi, l’output finale è JavaScript standard, più HTML e CSS, da pubblicare su hosting statico o piattaforme cloud.
-

Conclusioni: il nuovo web

- Alcuni esempi su <http://www.fabioitali.it/TW/2025/components/>
- La rivincita del modello computazionale su quello dichiarativo
- La **Component-Based Architecture** è qui per rimanere.
- HTML e CSS come l'assembler (ma... e i WebComponent?)
- Il ruolo delle aziende su WHATWG e W3C
 - Librerie e linguaggi proprietari: TypeScript, JSX, ecc.
 - CDN e posizionamento fisico delle librerie
 - La conquista delle teste degli sviluppatori



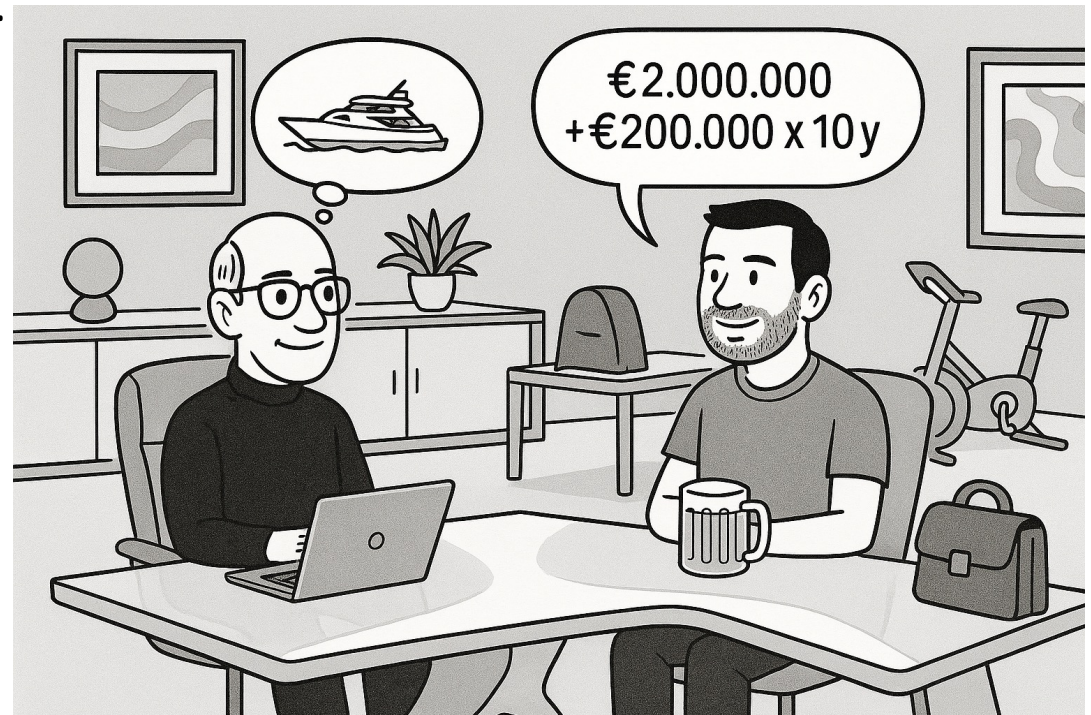
Il business del fornitore

Il contratto è interessante e fondamentale per l'azienda fornitrice, ma il budget iniziale **NON E'** la cosa più importante.

Un cliente di una certa dimensione, per un software di una certa importanza, metterà sicuramente a budget dei **contratti di manutenzione**, anche molto lunghi, dopo la fine della fase di realizzazione.

Se l'azienda fornitrice ha lavorato bene e il cliente è soddisfatto, è lei il candidato naturale a vincere il contratto di manutenzione.

- **Architettura del codice sofisticata e complessa** = l'azienda sviluppatrice è avvantaggiata a vincere il contratto di manutenzione.
- **Riuso della stessa libreria di componenti tra più progetti** = distribuzione dei costi di manutenzione tra molti fonti di entrate, con notevoli risparmi.

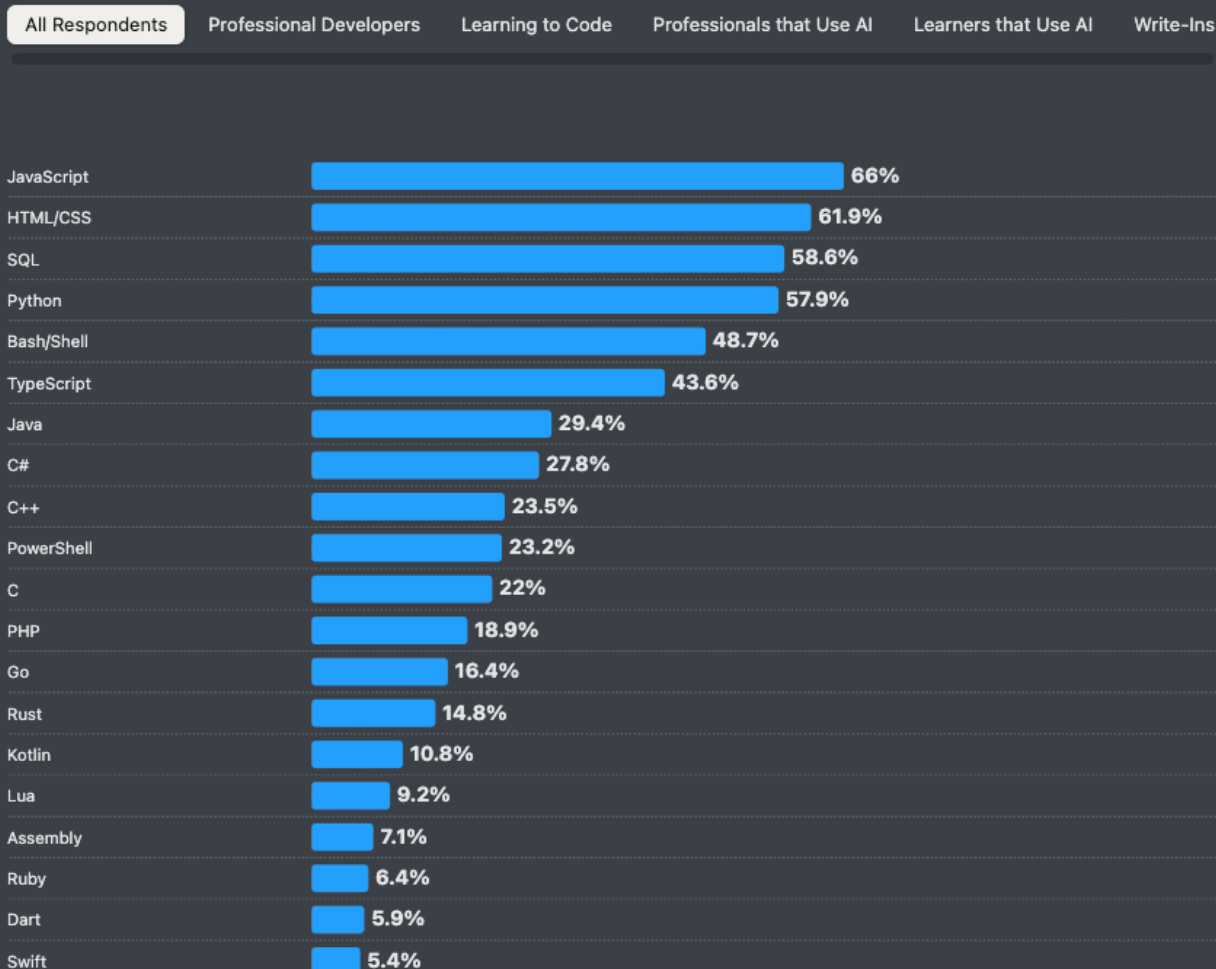


Un po' di statistiche

Programming, scripting, and markup languages

After more than a decade of steady growth, Python's adoption has accelerated significantly. It saw a 7 percentage point increase from 2024 to 2025; this speaks to its ability to be the go-to language for AI, data science, and back-end development.

? Which **programming, scripting, and markup languages** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)



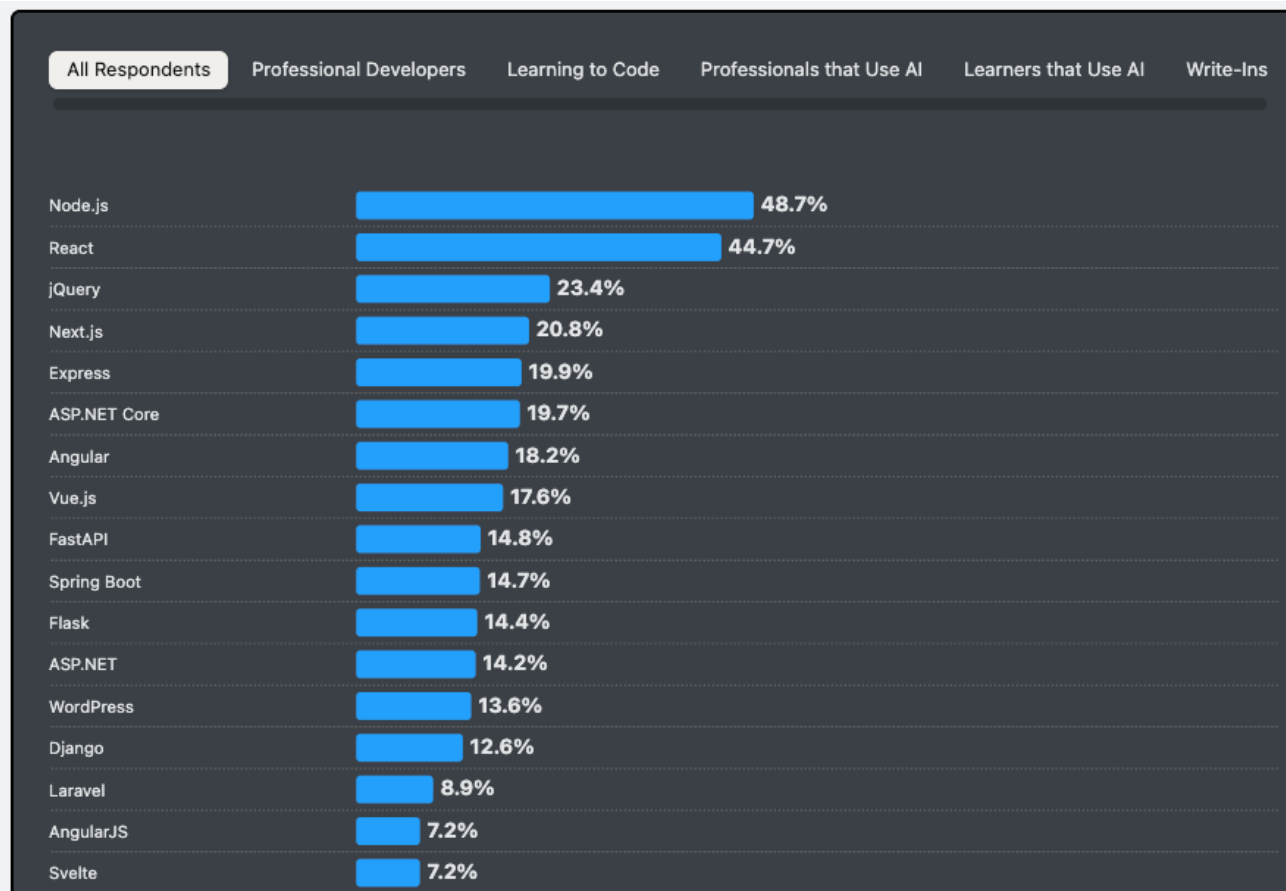
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Un po' di statistiche

Web frameworks and technologies

The +5 point increase for FastAPI is one of the most significant shifts in the web framework space. This signals a strong trend towards using Python for building performant APIs and reflects the overall strength of the Python ecosystem.

? Which web frameworks and web technologies have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the framework and want to continue to do so, please check both boxes in that row.)

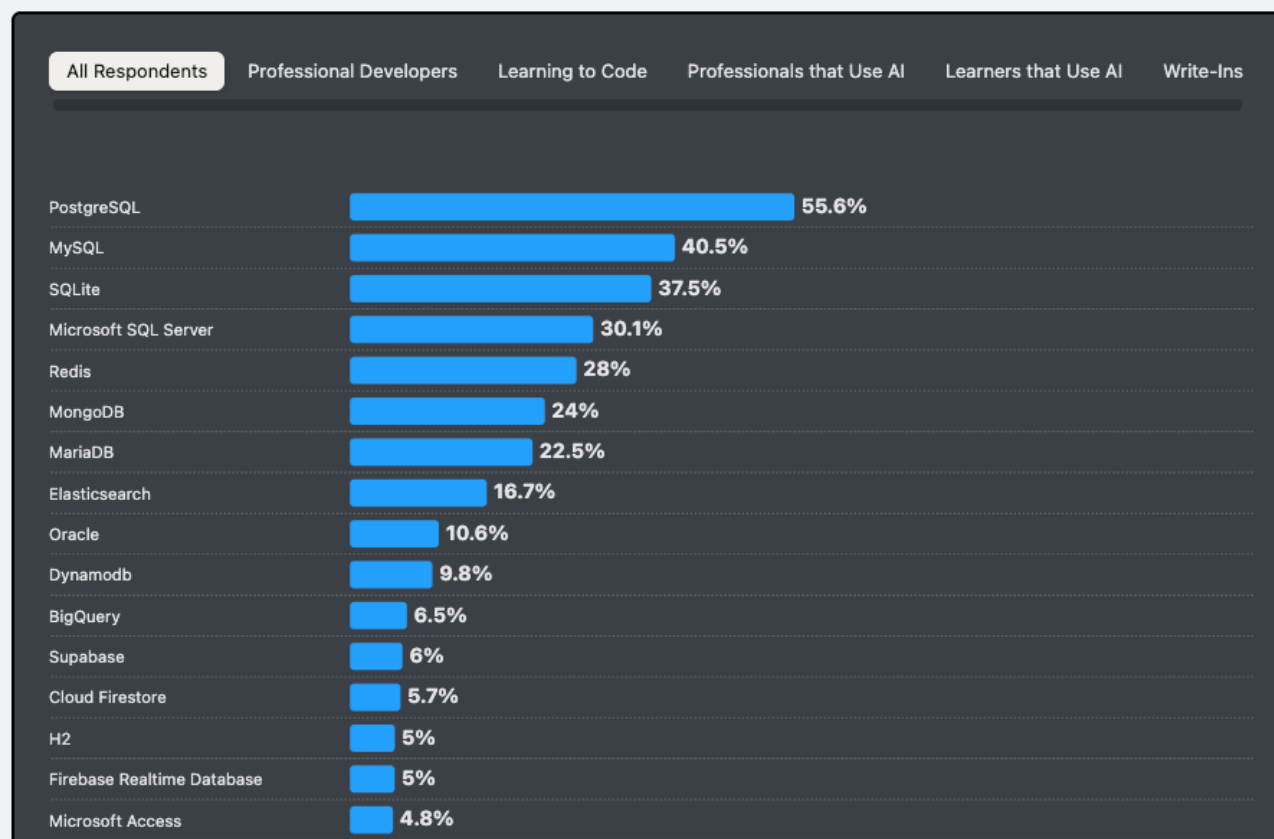


Un po' di statistiche

Databases

The significant growth in usage for Redis (+8%) highlights its growing importance. As applications become more complex, the need for high-speed, in-memory caching and data structures has made Redis an essential part of the modern tech stack.

? Which **database environments** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the database and want to continue to do so, please check both boxes in that row.)



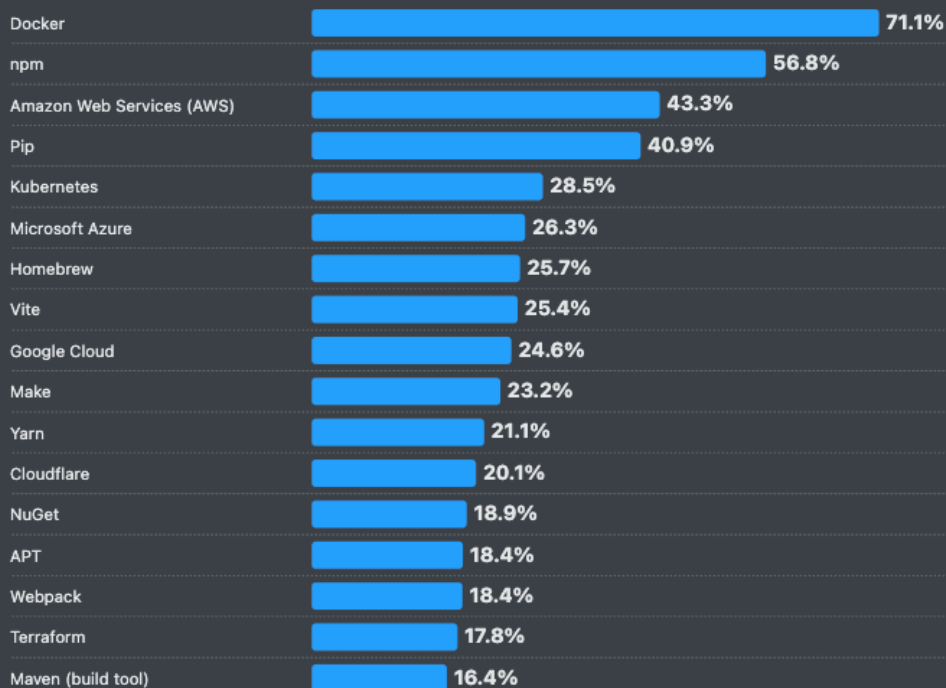
Un po' di statistiche

Cloud development

Docker has moved from a popular tool to a near-universal one. After years of growth, it experienced a +17 point jump in usage from 2024 to 2025, the largest single-year increase of any technology surveyed. This is partially due to consolidating some technology sectors in this year's survey.

? Which cloud platforms, containerization/orchestration tools, package managers, build tools, and infrastructure as code solutions have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the platform and want to continue to do so, please check both boxes in that row.)

All Respondents Professional Developers Learning to Code Professionals that Use AI Learners that Use AI



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Dipartimento di Informatica – Scienze e Ingegneria
Alma mater – Università di Bologna

Fabio.vitali@unibo.it

www.unibo.it